HPC Lab

Name: Aditi Mittal
Class: B.Tech CSE Final Year
PRN: 23610067
Batch: B1

Experiment no 1

Title: Introduction to HPC Environment and Shared Memory Parallel Programming

Objective: To understand the HPC environment and implement basic programs using OpenMP.

Problem Statement:

High Performance Computing (HPC) involves the use of parallel processing techniques to solve computationally intensive problems. Study basics of OpenMP for shared memory parallel programming using CPU threads.

Task 1: HPC Environment Setup and System

Configuration Perform the following activities:

1. Identify and record your system configuration:
    - Processor model: 11th Gen Intel(R) Core(TM) i3-1115G4 @ 3.00GHz
    - Number of CPU cores: 4
    - Number of logical processors: 4
    - RAM size: 8Gb

Linux Commands Used

Processor Information

lscpu

Number of Cores

nproc

RAM Information

free -h

```
lappy@box:~$ lscpu
Architecture:                    x86_64
  CPU op-mode(s):                32-bit, 64-bit
  Address sizes:                 39 bits physical, 48 bits virtual
  Byte Order:                    Little Endian
CPU(s):                          4
  On-line CPU(s) list:           0-3
Vendor ID:                       GenuineIntel
  Model name:                    11th Gen Intel(R) Core(TM) i3-1115G4 @ 3.00GHz
    CPU family:                  6
    Model:                       140
    Thread(s) per core:          2
    Core(s) per socket:          2
    Socket(s):                   1
    Stepping:                    1
    CPU max MHz:                 4100.0000
    CPU min MHz:                 400.0000
    BogoMIPS:                    5990.40
    Flags:                       fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pg
                                 e mca cmov pat pse36 clflush dts acpi mmx fxsr sse
                                 sse2 ss ht tm pbe syscall nx pdpe1gb rdtscp lm cons
                                 tant_tsc art arch_perfmon pebs bts rep_good nopl xt
                                 opology nonstop_tsc cpuid aperfmperf tsc_known_freq
                                  pni pclmulqdq dtes64 monitor ds_cpl vmx est tm2 ss
                                 se3 sdbg fma cx16 xtpr pdcm pcid sse4_1 sse4_2 x2ap
                                 ic movbe popcnt tsc_deadline_timer aes xsave avx f1
                                 6c rdrand lahf_lm abm 3dnowprefetch cpuid_fault epb
                                  cat_l2 cdp_l2 ssbd ibrs ibpb stibp ibrs_enhanced t
                                 pr_shadow flexpriority ept vpid ept_ad fsgsbase tsc
```

```
lappy@box:~$ nproc
4
```

```
lappy@box:~$ free -h
               total        used        free      shared  buff/cache   available
Mem:           7.4Gi       4.1Gi       685Mi       796Mi       2.6Gi       2.1Gi
Swap:             0B          0B          0B
```

2. Verify the installation using Linux OS and working of:
    - GCC compiler
    - OpenMP

Linux Commands Used

GCC Version

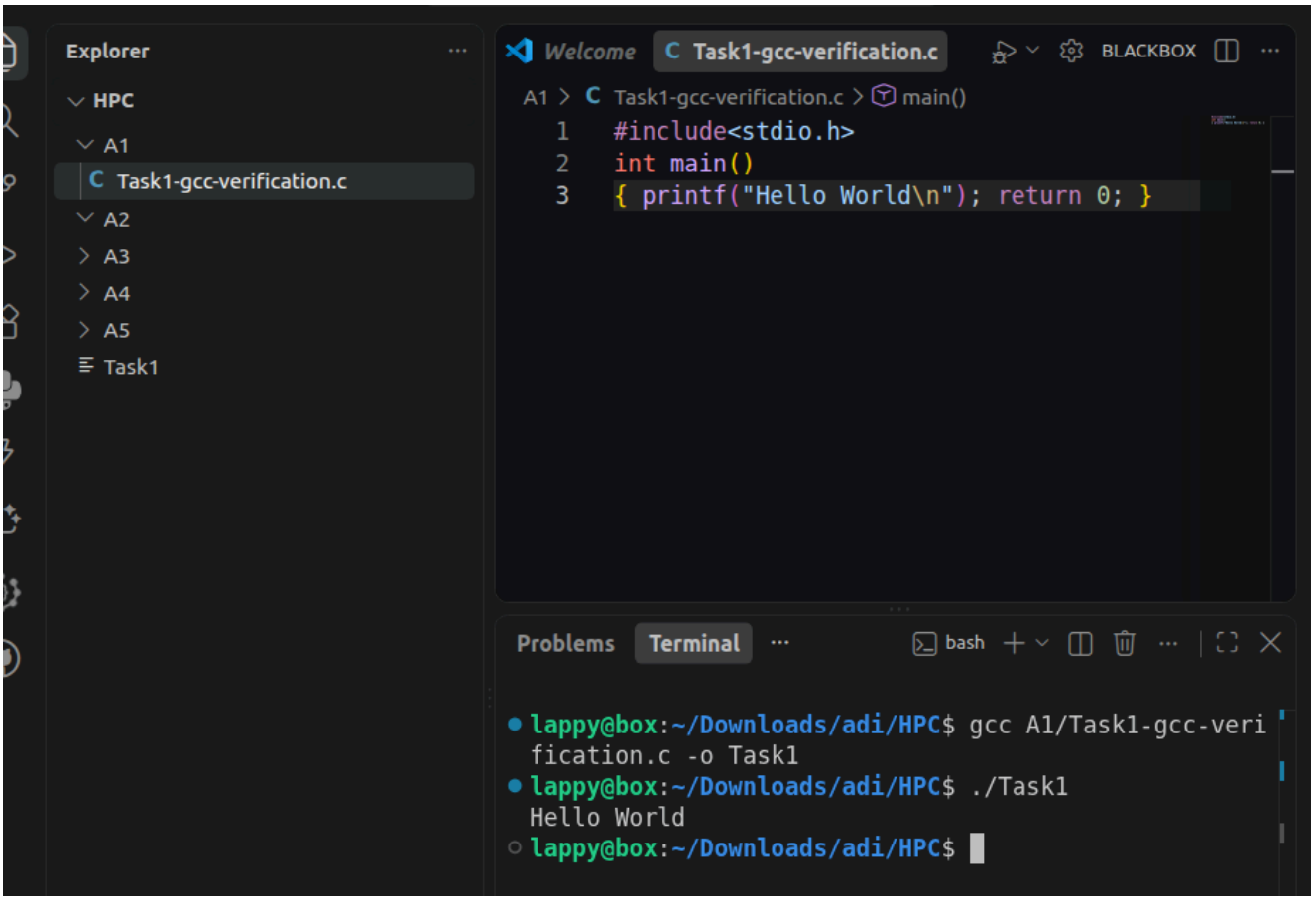
gcc --version

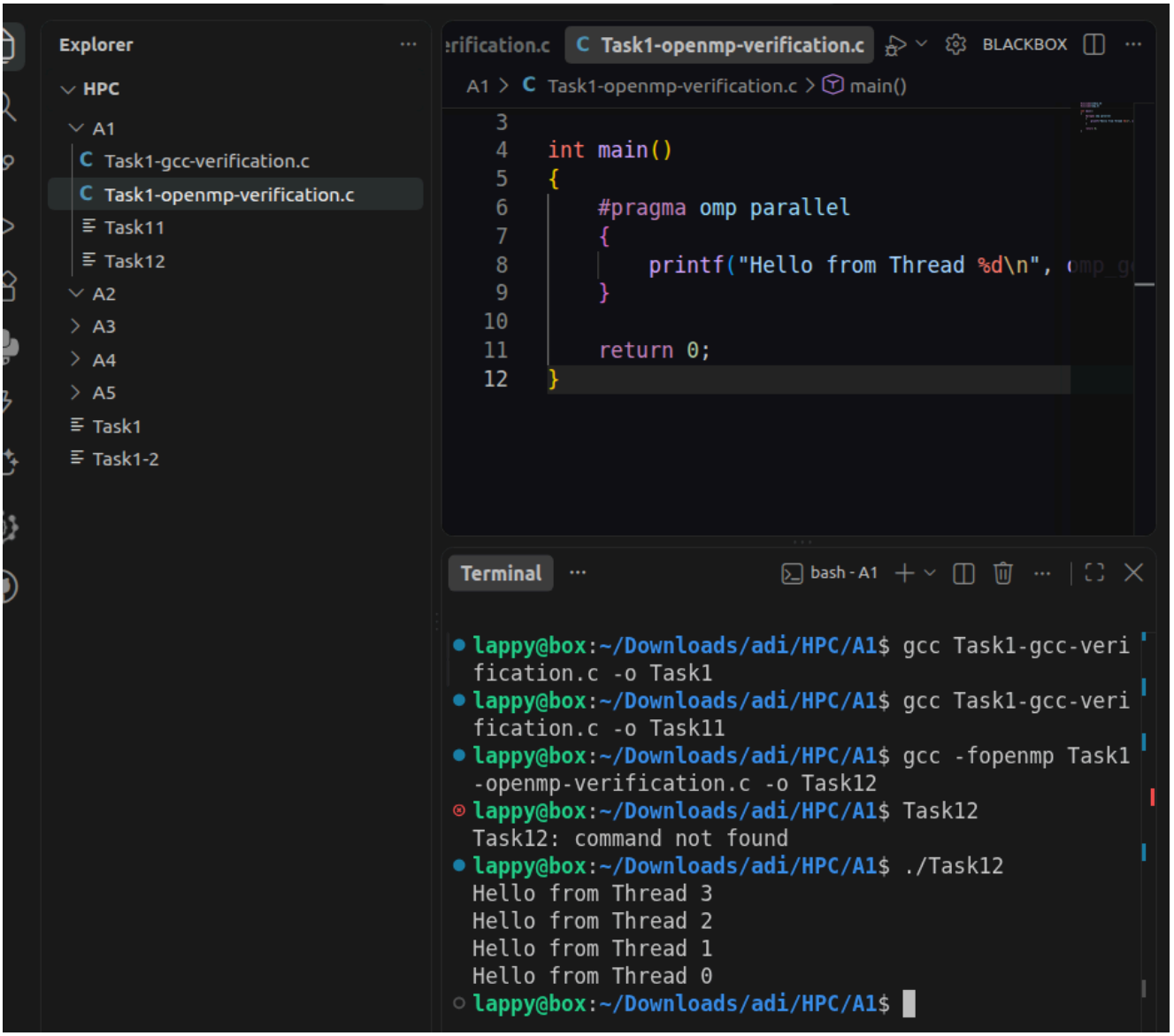
OpenMP Verification

gcc -fopenmp test.c -o test

```
lappy@box:~$ gcc --version
gcc (Ubuntu 11.4.0-1ubuntu1~22.04.3) 11.4.0
Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

3. Execute basic verification programs for each environment eg. Hello world program using multiple threads.





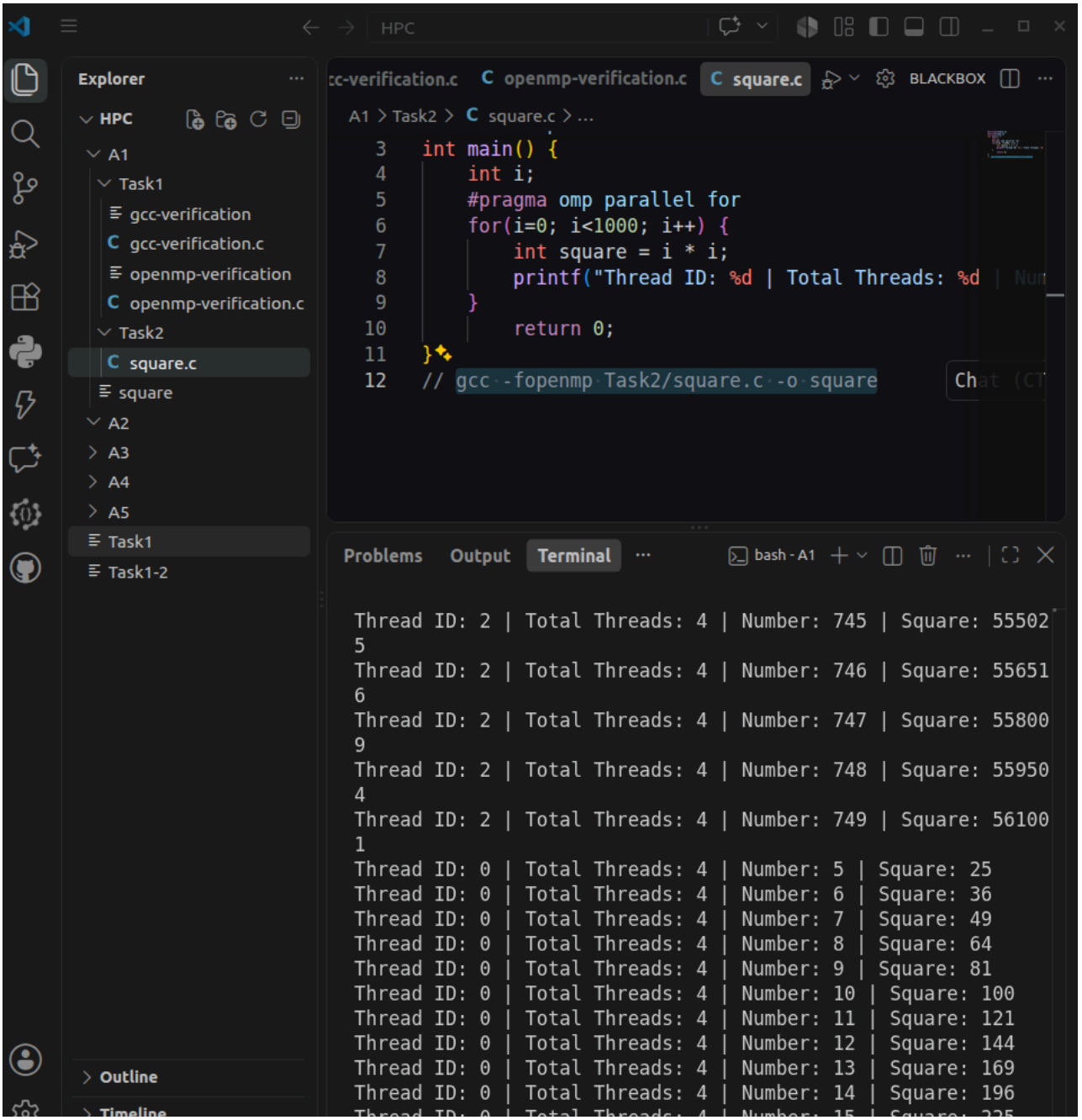
Task 2: OpenMP Thread Creation

Write an OpenMP program to create multiple threads and display thread information.

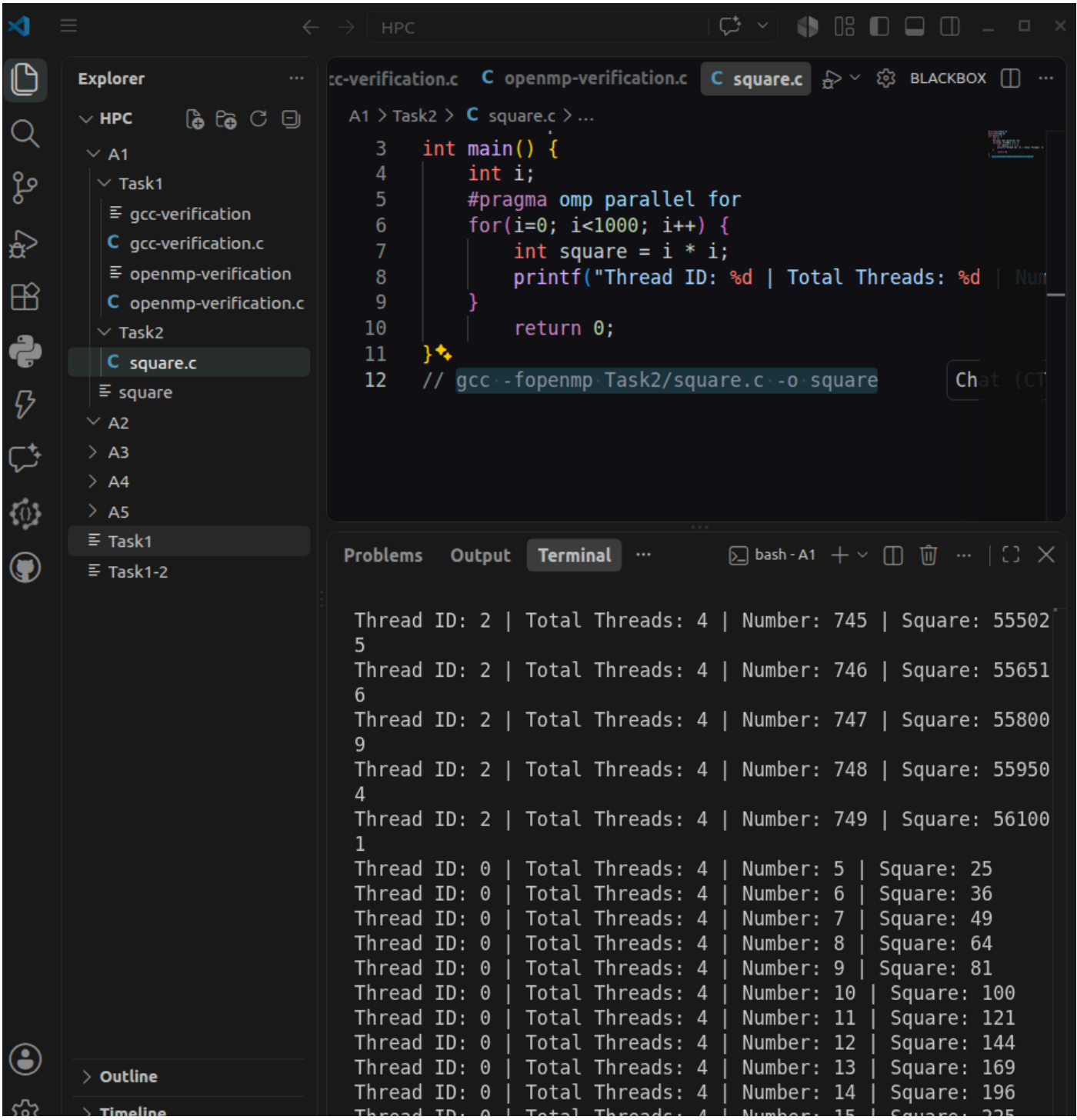
Requirements:

The program should:

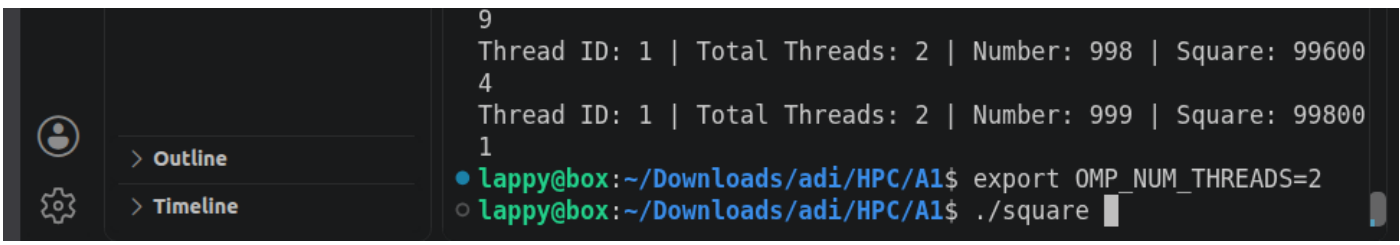
1. Create a parallel region using OpenMP for finding square of first 1000 whole numbers.



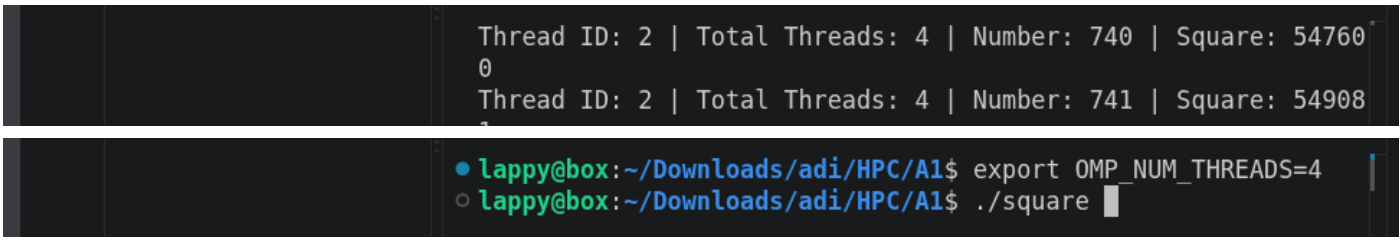
2. Display:
   - Thread ID
   - Total number of threads created



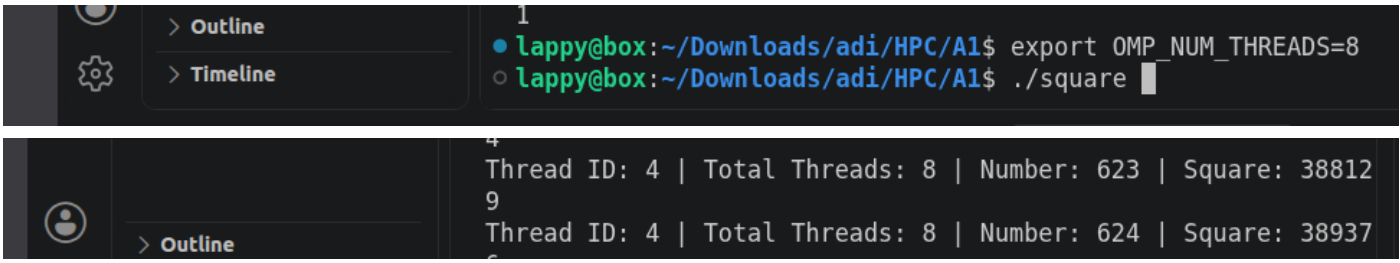
3. Execute the program using:
   - 2 threads



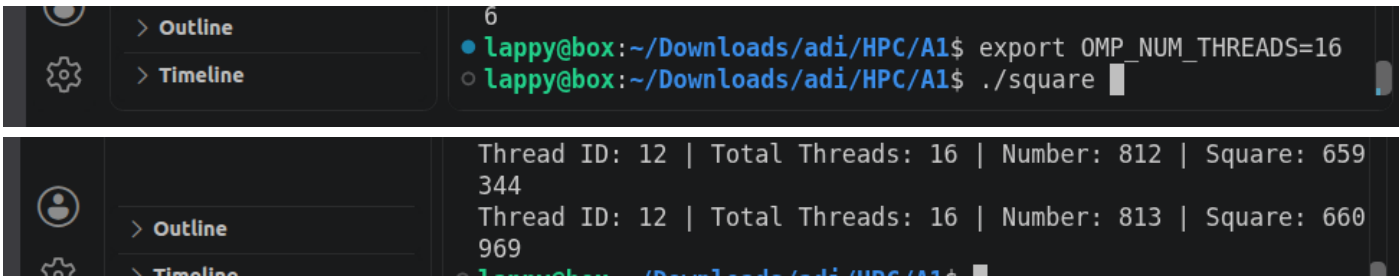
- 4 threads



- 8 threads



- 16 threads (if available)



Task 3: Calculate Theoretical Peak FLOPS of CPU

Theoretical CPU performance can be estimated using:

$$\text{Peak FLOPS} = \text{Number of Cores} \times \text{Clock Frequency} \times \text{FLOP/Clock}$$

where:

- Number of cores = Physical CPU cores
- Clock frequency = CPU operating frequency in Hz
- FLOP/Clock = Number of floating-point operations performed per clock cycle

For SIMD-based CPUs:

$$\text{FLOPS} = \text{Cores} \times \text{Frequency} \times \text{SIMD Width} \times \text{Operations per Instruction}$$

Where:

For Fused Multiply Add (FMA):

$$1 \text{ FMA} = 2 \text{ FLOPs}$$

(one multiplication + one addition)

Therefore:

$$\text{Peak FLOPS} = \text{Cores} \times \text{Frequency} \times \text{SIMD Width} \times 2$$

Compare the calculated flops with the manufacturers published Flops.

```
lappy@box:~/Downloads/adi/HPC$ /usr/bin/python3 /home/lappy/Downloads/adi/HPC/A1/Task3/flops.py
======================================================================
       THEORETICAL PEAK FLOPS CALCULATION
======================================================================

1. CPU INFORMATION
----------------------------------------------------------------------
Processor Model       : 11th Gen Intel(R) Core(TM) i3-1115G4 @ 3.00GHz
Physical CPU Cores    : 2
Logical Processors    : 4
Maximum Frequency     : 4.10 GHz
SIMD Instruction Set  : AVX-512
SIMD Register Width   : 512 bits
FP32 Values/Instruction: 16

2. VALUES USED
----------------------------------------------------------------------
Number of Cores       = 2
Frequency             = 4.10 GHz
Frequency in Hz       = 4.10e+09 Hz
SIMD Width            = 16 FP32 values
FMA                   = 2 FLOPs

3. FORMULA
----------------------------------------------------------------------
Peak FLOPS = Cores × Frequency × SIMD Width × Operations per Instruction

For FMA:
1 FMA = 2 FLOPs
Therefore:
Peak FLOPS = Cores × Frequency × SIMD Width × 2

4. SUBSTITUTION
----------------------------------------------------------------------
Peak FLOPS = 2 × 4.10e+09 × 16 × 2

5. CALCULATION
----------------------------------------------------------------------
2 × 4.10e+09
```

```
5. CALCULATION
----------------------------------------------------------------------
2 × 4.10e+09
= 8.20e+09

8.20e+09 × 16
= 1.31e+11

1.31e+11 × 2
= 2.62e+11 FLOPS

6. FINAL RESULT
======================================================================
Theoretical Peak Performance = 2.62e+11 FLOPS
Theoretical Peak Performance = 262.40 GFLOPS
Theoretical Peak Performance = 0.2624 TFLOPS
======================================================================

NOTE:
This is a theoretical peak calculation assuming:
• All physical CPU cores are operating at the selected frequency
• SIMD instructions are fully utilized
• FMA instructions are being used
• FP32 floating-point operations are being performed
Actual performance will normally be lower.
lappy@box:~/Downloads/adi/HPC$
```

The calculated theoretical peak of **262.4 GFLOPS** represents an ideal maximum under the assumption that both physical cores operate at 4.10 GHz and that 512-bit FP32 FMA instructions are fully utilized. Actual performance can be lower because of thermal limits, power limits, instruction mix, memory access, and CPU frequency changes during execution.

Result

- Studied the HPC environment and system configuration.
- Verified GCC compiler and OpenMP installation.
- Implemented OpenMP programs for parallel execution.
- Executed the program using different thread counts.
- Calculated the theoretical peak FLOPS of the CPU and compared it with manufacturer specifications.

Conclusion

This experiment introduced the fundamentals of High Performance Computing (HPC) and shared memory parallel programming using OpenMP. Parallel execution using multiple threads improved computational efficiency. Theoretical peak FLOPS were calculated to estimate CPU performance and compared with manufacturer values.